

# AI and Advanced Machine Learning

## miniproject

### Main idea

Imagining 2D pictures as 3D representations of the real world is easy for humans because we live in a 3D world and perceive the world in 3D. When presented with an image it is therefore clear for humans how that image is a 2D representation of a 3D scene.

For computers this task is rather complex as they perceive images as a matrix of rgb color codes and rather than a squashed 3D scene.

When an image is taken a 3D real world scene is squashed into a 2D frame, and the depth dimension is lost, to recreate this depth (or to generate it from scratch) qualified guesses depending on the 2D frame needs to be made.

With ChatGPT and other LLM utilizing the transformer architecture to illustrate sensible sentences based on rational guesses of next word, the idea in this miniproject was to apply this transformer architecture to solve this problem of predicting a 3D scene based on a 2D image as this problem also needs qualified guesses depending on the input to get the desired result.

### The problem statement

General problem statement was:

***How to implement a transformer to train a model to generate a 3D scene based on a 2D image.***

### Algorithm

#### Disclaimer:

Since I have limited resources and time to train my model accurately to perform well on real life scenarios I decided to use a simple case for the model of a mesh background color and some generated shapes for the training. This was to ensure that limited training would give some meaningful results

## Algorithm overview:

I used an encoder decoder transformer algorithm to train my model. I used pytorch modules for the models and implemented my own subclasses for the pytorch model classes.

The complete workflow of the transformer is provided in the README file inside the project. But in general it goes something like this:

For each iteration of the training loop:

1. Data generation: Generates synthetic data for the model to train on
2. Patch embedding to generate tokens over the input image
3. Adding positional embeddings to each token depending on their location in the image
4. Transformer block initialization
  - Starts a fixed amount of transformer blocks, each block performs:
    - Normalization,
    - Multihead self attention across blocks
    - Self attention heads inside each block
    - Apply attention using a residual connection
    - Feed forward MLP to process each token after attention is applied to them
5. After transformer blocks as runned the tokens gets normalized again
6. Reshapes image back to 2D
7. CNN upsampling back to original image size
8. Computes loss and updates weights for next iterations

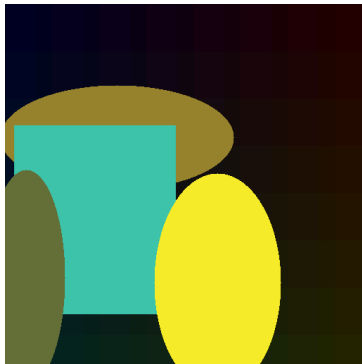
The model weights are saved as pth files inside the model subfolder. These can be used using the [predict.py](#) script to apply the model to an image and see the model guess the 3D scene.

## Results and discussion

The model shows that it can correctly identify edges of objects and project them away from the background surface. However the model lacks understanding of the center pieces of the objects. This is likely something that can be improved by tweaking patch size to always have an object edge in the frame. On real life pictures the model performs worse as it lacks complex image data to train on and only can relate to simple objects.

Result plots:

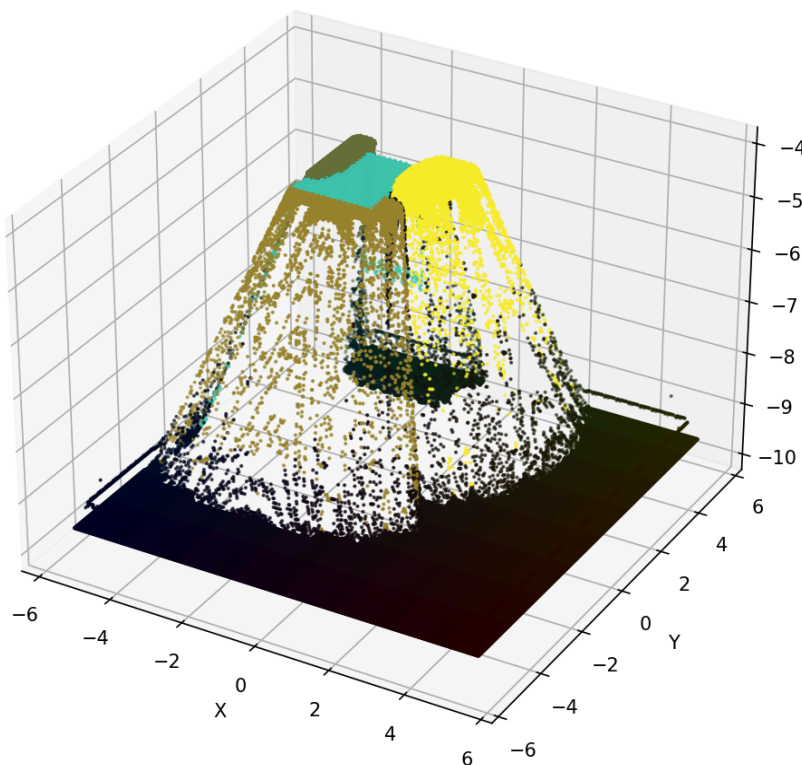
Input image:



Output image:



3D output estimation:



## Reflections

This project shows that Transformers are a valid technology for building models that predict lost or unknown information. The field of transformers are constantly evolving, and features like Dropout and Cross attention mechanisms are a welcomed tool to support continuous improvements in the field

The results show that building a model to predict a 3D render is a complex task that requires extensive amounts of training and data to perform more reliably than this current version.

To extend this version of the transformer to generate models able to predict real photos 3D render is an even more complex task as the level of detail increases exponentially compared to the dataset this model was exposed to.

## Conclusion

To answer the problem statement:

***How to implement a transformer to train a model to generate a 3D scene based on a 2D image.***

The design i produces suffices to show a valid starting point for an approach. Mixing CNN encoding mechanisms and multihead attention indicates that the transformer architecture is a valid approach to solve this problem. However much work is still needed for this particular algorithm and additional training to see its full potential.